

Экзамен: «По сусекам» за 2–2,5 часа

Итог репетиции 23.09.2026. Три части:

- 1. **Подготовка до экзамена** — всё, что делается руками и отнимает время.
- 2. **Первый промпт** — вставить в Claude Code в пустой папке.
- 3. **Спецификация** (SPEC ниже) — финальная версия со всеми решениями, принятыми на репетиции. Её кладём в папку проекта как `CLAUDE.md`, чтобы Claude не переспрашивал и не переделывал.

Рабочий пример и журнал с граблями: `D:\ClaudeProjects\FigmaTestVueJS (EXAM-NOTES.md)`.

1. Подготовка — сделать заранее, дома (≈ 20 минут)

#	Что	Как проверить
1	Ключ Anthropic, месячный лимит расходов в Console → Limits (auto-reload не включать)	ключ <code>sk-ant-...</code> есть
2	Ключ Unsplash: <code>unsplash.com/developers</code> → Register as developer → <code>unsplash.com/oauth/applications</code> → New Application → 4 галочки → Create → Access Key . Для экзамена — отдельное приложение , чтобы тесты не сожгли его 50 запросов/час	ключ есть; до показа им не пользоваться
26	Ключ Pexels: <code>pexels.com</code> → «Я хочу скачивать контент» → <code>pexels.com/api/key</code> : Project Name, Category Lifestyle , описание («Recipe web app... photos illustrate recipe cards, with attribution»), URL, галочка → ключ сразу	ключ есть
3	<code>npx wrangler login</code> уже выполнен на экзаменационном компьютере	<code>npx wrangler whoami</code> показывает аккаунт
4	Node 20+, Chrome, Claude Code, Python — установлены	<code>node -v</code>
5	Файл <code>.dev.vars</code> с ключами лежит вне папки проекта, например <code>D:\exam-keys\.dev.vars</code>	ключи в чат не вставлять
6	Разрешения Claude Code: auto mode или allowlist для <code>npm</code> , <code>npx</code> , <code>node</code> , <code>curl</code> , <code>python</code>	запросы на разрешение не отвлекают
7	Имя Cloudflare-проекта po-susekam свободно (тестовый был <code>po-susekam-test</code>)	<code>npx wrangler pages project list</code>

Формат `.dev.vars`:

```
ANTHROPIC_API_KEY=sk-ant-...
UNSPLASH_ACCESS_KEY=...
PEXELS_API_KEY=...
```

Каждая строка — с начала, **без пробела перед именем** (на репетиции лишний пробел спрятал ключ Pexels).

2. Как начать на экзамене

Основной способ — скилл `/po-susekam`. Лежит в `C:\Users\DIVO\.claude\skills\po-susekam`: SPEC, готовые стили, лого, скрипт самопроверки, эталонный код. На этом компьютере уже установлен. На другом компьютере — скопировать эту папку в `%USERPROFILE%\.claude\skills\`.

1. Создать пустую папку, положить в неё `.dev.vars` с тремя ключами.
2. Открыть в ней Claude Code, включить auto mode.
3. Написать одну строку:

```
/po-susekam имя=po-susekam
```

Если правила экзамена разрешают свой готовый код — `/po-susekam имя=po-susekam --with-reference` (быстрее, и вёрстка один в один с репетицией).

Дальше Claude сам: проверит ключи и логин, скопирует SPEC в `CLAUDE.md`, пройдёт этапы 0–9 с самопроверкой и отчётом после каждого, задеплоит и пришлёт адрес.

Запасной способ — без скилла: положить `CLAUDE.md` = раздел `SPEC` и вставить промпт ниже.

Промпт для запасного способа

Создать пустую папку, положить туда `CLAUDE.md` = раздел `SPEC` целиком, скопировать `.dev.vars`, запустить Claude Code и вставить:

Собираем проект «По сусекам» по `CLAUDE.md` в этой папке – это полная спецификация, все решения уже приняты, не переспрашивай и не предлагай альтернатив. Ключи лежат в `.dev.vars` (не выводи их), wrangler уже залогинен.

Работай этапами из раздела «План» по порядку. После каждого этапа:

- ``npm run build`` без ошибок;
- проверь этап сам: `curl` к API (JSON-запросы – из UTF-8 файла через `--data-binary @file`, не `-d`) и headless Chrome через CDP-скрипт на Node (скриншоты 1536×730 и 390×800, без мобильной эмуляции);
- одной строкой напиши мне: этап, что проверено, сколько минут ушло.

Не останавливайся между этапами и не жди подтверждения – у тебя полное разрешение, включая деплой в конце под именем `po-susekam`. Если что-то не получается дольше 10 минут – упрости и иди дальше, в конце перечисли, что упростил.

Дальше только смотреть и нажимать «разрешить», если что-то спросит.

3. Бюджет времени (по замерам репетиции)

Этап	Репетиция	План
0. Каркас Vite + Vue + wrangler + стили	—	10 мин
1. <code>/api/recipes</code> + structured outputs	3 мин	10 мин
2. Поиск: чипсы, карусель, стор	6 мин	15 мин
3. Фото Unsplash + страница рецепта	3 мин	10 мин

Этап	Репетиция	План
4. Фото продуктов → распознавание	3 мин	10 мин
5. Голос → распознавание	~10 мин	10 мин
6. «Ещё рецепты» + лимит, калории	~15 мин	15 мин
7. Пошаговый режим + YouTube	~15 мин	15 мин
8. Избранное	~10 мин	10 мин
9. Деплой (проект 6 с, секреты 10 с, deploy 15 с) + проверка прода	2 мин	10 мин
Запас на грабли		20–35 мин
Итого		≈ 2 ч – 2 ч 15 мин

Если время поджимает, резать в таком порядке: YouTube-кнопку оставить, пошаговый режим упростить (без таймеров), калории оставить (дёшево), избранное — последним.

СПЕС — положить в папку проекта как `CLAUDE.md`

```
# По сусекам — рецепты из того, что есть в холодильнике

Интерфейс и все тексты — на русском. Код — Vue 3 **Options API**, Vite, SCSS. Комментарии в коде — по-русски, объясняют «почему».

## Архитектура
Vue 3 + Vite — /api/* → Cloudflare Pages Functions (functions/api/*.js) → Claude API / Unsplash
- Один Pages-проект: фронт в dist/, функции рядом. Локально: `npm run dev` (Vite 5173, проху /api → 8788)
  и `npm run dev:api` = `wrangler pages dev --port 8788` (перед первым запуском `npm run build`).
- wrangler.toml: name = "po-susekam", pages_build_output_dir = "dist", compatibility_flags = ["nodejs_compat"],
  [vars] RECIPES_COUNT = "6", SEARCH_ROUNDS = "2" (поиск + одно «Ещё рецепты» = 12 рецептов и 12 фото — бережём лимит Unsplash). Секреты — .dev.vars (в .gitignore вместе с .wrangler).
- Модель: `claude-sonnet-5`, `output_config: { effort: 'low', format: { type: 'json_schema', schema } }`.
  Логировать `response.usage`. Обработать refusal, max_tokens, RateLimitError (429), APIError (502).
- Зависимости: vue, vue-router, @anthropic-ai/sdk; dev: vite, @vitejs/plugin-vue, sass, wrangler.

## Дизайн
- Цвета: accent #ffde67, cream #fff4cb, rust #9f4620, maroon #3c0202 (фон страниц), black #000, neutral-800 #2c2b2b.
- Шрифты: Inter (текст), Playfair Display 800 (лого и заголовок) — Google Fonts, есть кириллица.
- Колонка контента: миксин content-column — max-width 960px, поля 32px (≤600px — 20px). По ней выровнены
  хедер, поиск, заголовки, рецепт; карусель начинается на той же вертикали:
  padding-inline: max(32px, calc((100% - 960px) / 2 + 32px)).
- Компактно: всё должно влезать в 1536×730 CSS px (ноутбук 1080p с масштабом 125%). Заголовок h1 24px (моб. 22),
  подпись 14px, поле и кнопка по 44px, карточка 280px шириной, фото карточки 120px, заголовок карточки 16px, текст 13px.
- Брейкпоинты: tablet ≤1024, mobile ≤600. Без горизонтального скrolла на 390px.

## Хедер и футер
- Липкий хедер (sticky), чёрный; при прокрутке >8px — rgba(0,0,0,.72) + backdrop-filter blur(10px) + тень.
```

Полоска rust брх сверху.

- Слева: SVG-лого (жёлтый холодильник в белом поварском колпаке, глаза и улыбка), «По сусекам» (Playfair 19px),

слоган «Шедевры из того, что есть в холодильнике» (11px). Лого – ссылка на главную.

- Справа: «← Назад к рецептам» (на рецепте и в избранном; с рецепта из избранного – «← Назад к избранному»;

на телефоне просто «← Назад», слоган скрыт) и «♥ Избранное» + жёлтый счётчик (на телефоне – сердце и счётчик).

- Футер: «© {текущий год} «По сусекам». Все права защищены.» Никаких соцсетей, меню и поиска в хедере.

- Круглая жёлтая кнопка «наверх» справа внизу после 400px прокрутки (prefers-reduced-motion – без плавности).

Главная

- h1 «Что есть в холодильнике?», подпись «Напишите, продиктуйте или сфотографируйте продукты – подберём блюда...».

- Поле на всю ширину колонки: чипсы продуктов (запятая/Enter добавляют, Backspace в пустом поле удаляет последний,

дубли без учёта регистра отсекаются, максимум 30, всё в нижнем регистре). Внутри поля справа иконки без подписей:

- × «Очистить» (только если есть продукты), 📷 камера (label + input file accept=image/* capture=environment),

- 🗣️ микрофон (только если есть Web Speech API). На телефоне при наличии продуктов иконки уходят в нижний ряд поля.

У input `width: 0; min-width: 140px`, у панели `width: 100%` – иначе на 390px вылезает за экран.

- Под полем – кнопка «Подобрать блюда» на всю ширину. Поиск запускается ТОЛЬКО этой кнопкой (или Enter в пустом поле).

- Голос и фото только ДОПИСЫВАЮТ продукты в поле и пишут «Добавили: Можно добавить ещё или нажать «Подобрать блюда»».

Съедобного не нашлось – «Съедобных продуктов не нашли – назовите или сфотографируйте продукты»;

всё уже в поле – «Эти продукты уже в списке – можно добавить ещё или нажать «Подобрать блюда»».

- Ничего не стирается само (ни фокус, ни микрофон, ни камера). Стирает только × в поле: продукты, рецепты, фразу,

превью. Если удалить последний чипс – рецепты тоже уходят (watch; в resetSearch присваивать [] только если не пусто).

- Пока ничего не искали – только поле и «Попробуйте: картошка, сосиски, помидоры / яйца, молоко, сыр / курица, рис, морковь». Никаких моковых карточек.

- Во время поиска: 3 скелетона, «Обычно это занимает 15–25 секунд». Пустой ответ – «Из этих продуктов не

получилось подобрать блюда...».

- Результаты: заголовок «Что можно приготовить», горизонтальная карусель (scroll-snap, тонкий скроллбар брх,

drag мышью без случайного клика по ссылке). Под каруселью – кнопка «Ещё рецепты».

- Состояние поиска (продукты, результаты, canLoadMore) – reactive-стоп + sessionStorage: переживает F5 и переход на рецепт.

Карточка рецепта

- Фото (или градиент rust→maroon с 🍷), название, описание, строка «🕒 30 мин · ≈ 610 ккал · не хватает: лук» /

«всё есть», кнопка «Смотреть рецепт».

- Кликается ВСЯ карточка: кнопка-ссылка с ::after { position:absolute; inset:0 }, у карточки position:relative.

Hover – приподнять на 2px + тень. Фокус обводит всю карточку (rust).

- Сердце ♥ в правом верхнем углу фото (над растянутой ссылкой, click.stop.prevent) – избранное.

API

```
### POST /api/recipes { ingredients: string[], exclude?: string[] }
```

- Системный промпт: шеф домашней кухни, «Предложи \${RECIPES_COUNT} разных блюд – от простых и быстрых до чуть более

интересных»; соль/перец/масло/вода/сахар всегда есть; missing – чего не хватает; шаги без нумерации; image_query – 2–4 слова по-английски; ingredient_calories – ккал каждого пункта ingredients в том же порядке

(макароны/крупы/мука по сухому весу, соль/специи/вода – 0); несъедобное рядом со съедобным молча пропустить

и готовить из остального (даже если съедобный один); пустой recipes – только если съедобного нет совсем.

- JSON-схема рецепта (порядок полей ВАЖЕН – модель пишет по порядку): title, description, time_minutes, servings, ingredients, ingredient_calories, missing, steps, tip, image_query. additionalProperties: false.
- Калории на порцию считает СЕРБЕР: сумма ingredient_calories / servings, округлить до 10 → calories_per_serving.

(Модель на effort low складывает на глаз и занижает на треть.)

- exclude – названия уже показанных блюд: «Эти блюда я уже видел, предложи другие...».
- Лимит раундов: round = floor(exclude.length / count) + 1; если round > SEARCH_ROUNDS → 429 { limitReached: true }

без вызова Claude. Ответ: { recipes: recipes.slice(0, count), canLoadMore: round < SEARCH_ROUNDS }.

POST /api/recognize { image, mediaType } | { text }

- Фото (jpeg/png/webp/gif, ≤5 МБ base64) или продиктованная фраза (≤1000 символов) → { ingredients: string[] }.

Общие правила: по-русски, нижний регистр, общие названия, без количества, не выдумывать.

Для текста: «распознано автоматически, без препинания и с ошибками – исправь явные ошибки».

GET /api/photo?q=

- Сначала Unsplash search/photos (per_page 5, orientation landscape, content_filter high, запрос `\${q} food`),
- если отказал или пусто – Pexels /v1/search (per_page 5, orientation landscape, заголовок Authorization: ключ).

Ответ { photos: [{ id: 'unsplash-...' | 'pexels-...', raw, alt, author, authorUrl, sourceName, sourceUrl }] };

у Unsplash – utm_source в ссылках, у Pexels sourceUrl = страница фото. Ресайз у обоих параметрами ссылки.

Кэш caches.default на сутки. Нет ключей → { photos: [] }. Оба отказали → 503, НЕ кэшировать.

- Клиент берёт первое фото, чей **id** (не raw – в raw свой ixid) не занят другими карточками. При ошибке

photoStatus остаётся idle – догрузится при следующем заходе. Размер через параметры raw: w, h, fit=crop, q=75.

- Подпись на странице рецепта «Фото: автор / Unsplash|Pexels» со ссылками – обязательна по правилам обоих.

Фото и голос на клиенте

- prepareImage: createImageBitmap → canvas ≤1568px → JPEG 0.85 → base64 + превью. HEIC не открылся – «Попробуйте JPG или PNG». После выбора файла event.target.value = ''.
- Голос: webkitSpeechRecognition, ru-RU, continuous + interimResults, фраза видна по мере речи, стоп после 2,5 с тишины или 20 с, повторное нажатие – «Готово». Ошибки: not-allowed, audio-capture, no-speech, network – по-русски.

Во время записи микрофон rust с пульсацией, остальное заблокировано.

«Ещё рецепты»

- Те же продукты + exclude; новые карточки в конец карусели, пока ждём – 3 скелетона в конце, карусель прокручивается к первой новой карточке (и ещё раз после загрузки). Всего 2 раунда × 6 = 12.
- Лимит исчерпан: кнопка disabled, рамка и текст почти прозрачные (cream 25% / 30%), cursor not-allowed, рядом: «Лимит подбора исчерпан – выберите из этих рецептов или измените продукты и начните новый поиск».

Страница рецепта /recipes/:slug

- Колонка 960px, текст ≤720px. Фото 16:9, max-height 360px, скругление; на фото снизу (под градиентом) кнопки:

«► Готовить по шагам» (жёлтая), «Видео на YouTube» (красная плашка; ссылка на `youtube.com/results?search_query=<название> рецепт, target=_blank` — поиск, а не конкретный ролик), «♥ В избранное / ♥ В избранном».

- Заголовок 26px, описание, плашки: 🕒 мин, 🍴 порц., 🔥 ≈ N ккал на порцию, Не хватает / Все продукты есть.

Ингредиенты, «Приготовление» (нумерованный список), «Совет».

- Рецепт ищется: результаты поиска → избранное (slug fav-...).

Пошаговый режим (CookingMode.vue)

- ``<dialog>`` + `showModal()` на весь экран (100vw × 100dvh), фон — размытое фото блюда. Шапка: название, мини-таймеры

других шагов, ×. «Шаг 2 из 5 · Жарка» + полоска прогресса. Один шаг на экран: иконка 64px, текст 24px (моб. 20).

Назад / Далее (последний: «Готово — приятного аппетита!»), ↔ на клавиатуре, свайп, Esc закрывает.

- Иконка и таймер — из текста шага регулярками (без Claude): 🔥 запек/духовк/выпека; 🍳 жар/сковород; 🍲 вар/кипя/туш/нагре; 🍴 нареж/поруб/натр; 🍹 смеша/перемеш/взбе/блендер; 🍴 подава/разле/стопк/полейте;

🍴 посол/посып. Таймер: «8–10 минут» → 8:00 (меньшее), час/мин/сек, «полчаса», «минуту» —

`**\b` в JS не работает с кириллицей**, границы через (`?<![а-яё]`).

- Несколько таймеров одновременно (`timers[step]`); время от `endTime`, а не тиками. `AudioContext` создаётся по нажатию

«Запустить таймер» (Safari), в конце 3 сигнала + `navigator.vibrate` + «Время вышло!». `Wake Lock`, пока окно открыто.

На последнем шаге — совет.

Избранное (localStorage)

- Стор favorites: рецепт сохраняется ЦЕЛИКОМ (копия), slug = fav-`<хэш названия>`, дубли по названию склеиваются,

синхронизация вкладок через событие `storage`, максимум 100.

- /favorites: «Избранное», «N рецептов · сохранены в этом браузере», сетка карточек (auto-fill, minmax 260px),

пусто — «Здесь пока пусто. Нажмите ♥ на фото рецепта...» + ссылка «Подобрать рецепты».

План (этапы)

0. Каркас: Vite + Vue + router + SCSS-переменные и миксины, `wrangler.toml`, `.gitignore`, хедер/футер/кнопка «наверх».

1. /api/recipes (схема, калории, exclude, лимит) → проверить curl из UTF-8 файла.

2. Главная: поле с чипсами и иконками, кнопка, стор, карусель, карточка, скелетоны.

3. /api/photo + уникальные фото + страница рецепта.

4. /api/recognize (фото) + камера.

5. Голос (тот же /api/recognize с text).

6. «Ещё рецепты» + лимит.

7. Пошаговый режим + YouTube.

8. Избранное.

9. Деплой: ``npx wrangler pages project create po-susekam --production-branch main``; 3 секрета из `.dev.vars` через `stdin`

``wrangler pages secret put NAME --project-name po-susekam``; ``npm run build``;

``npx wrangler pages deploy dist --project-name po-susekam --branch main --commit-dirty=true``;

проверить прод.

Грабли Windows

- curl из Git Bash портит кириллицу в -d → JSON только из UTF-8 файла через `--data-binary @file`.

- Python print кириллицы → `PYTHONIOENCODING=utf-8`.

- Остановка wrangler не всегда убивает `workerd.exe`: порт 8788 занят, запросы висят. Проверка: `Get-NetTCPConnection -LocalPort 8788`; останавливать всё дерево (`node wrangler + workerd`).

- После правки `.dev.vars` или `wrangler.toml` — перезапуск ``npm run dev:api``.

- Unsplash Demo — 50 запросов/час: на тестах кончается быстро (403 remaining 0).

- Расширение Chrome не снимает скриншоты свёрнутого окна — использовать headless Chrome + CDP (Node, global WebSocket).

Скриншот с прокруткой — без clip (clip в координатах документа).
- Vue срезает ведущий пробел в `<span> {{ x }}</span>` → ` {{ x }}`.

Если спросят про фото и лимиты (проверено 23.09.2026 по официальной документации)

Сервис	Бесплатно	Больше
Unsplash API	Demo: 50 запросов/час . После ручной проверки приложения (Production): 1 000/час — тоже бесплатно	платного тарифа нет; выше 1 000/час — индивидуально через partnerships@unsplash.com
Pexels API	200/час, 20 000/месяц	платного тарифа нет; лимит поднимают бесплатно по запросу, если показать использование с атрибуцией

Unsplash+ (платная подписка на сайте) — это лицензия на премиум-фото для людей, **не** лимиты API.

Ответ на экзамене: «API фото бесплатный. Сейчас демо-лимит 50 запросов в час — это ~4 полных сессии (поиск + „Ещё рецепты“ = 12 фото). Для заказчика: подаём приложение на Production в Unsplash (1 000/час бесплатно, ~80 сессий в час), добавляем Pexels запасным источником (+200/час), фото кэшируются на сутки на Cloudflare. Если нужно больше — договор с Unsplash напрямую. Платим только за Claude: ≈ \$0,02 за поиск».

Доработка вживую: голос во всех браузерах (цель ≈ 15 мин)

Когда основная сборка задеплоена — вторая команда в той же сессии и той же папке:

```
/po-susekam add-voice
```

Сценарий показа: открыть сайт в Firefox — микрофона нет → команда → Claude пишет `/api/transcribe` (Whisper на Cloudflare Workers AI) и запасную запись в браузере, сам проверяет (`voice-check.mjs`: запись, Brave, тишина) и деплоит в тот же проект → обновить страницу в Firefox, нажать микрофон, продиктовать продукты.

Всё, что нужно Claude, лежит в скилле: `VOICE.md` (задача и грабли), `scripts/voice-check.mjs` + `fixtures/`, `reference-voice/` (код, только с `--with-reference`).

Замеры репетиции 25.09: Whisper \$0,0005/мин, 10 000 нейронов/сутки бесплатно ≈ 640 диктовок по 20 с; распознавание 0,7–1,8 с; от клика до продуктов в поле ≈ 12–14 с (6 с речи + 2,5 с тишины + Whisper + Claude). Живая проверка: Firefox (десктоп) и iPhone — работает.

Evals: автопроверка ответов модели (29.09.2026)

Что это. Автотесты для ответов Claude. Модель на один вход отвечает каждый раз немного по-разному, поэтому проверяются не точные совпадения, а свойства ответа, и результат — процент («27 из 30»). Нужны, чтобы менять промпт, effort или модель и сразу видеть в цифрах, что стало лучше, а что сломалось. Ручная проверка одного кейса в браузере — тоже eval, только один раз, один кейс и без повтора после каждой правки.

Где лежит (репозиторий po-susekam, папка `evals/`):

Файл	Что внутри
<code>cases.mjs</code>	18 кейсов: обычные холодильники, один продукт, 12 продуктов, опечатки, английский, несъедобное (камни, гвозди, мусор), несъедобное рядом со съедобным (камни + яйца, бумага + клей + сыр), <code>exclude</code>
<code>checks.mjs</code>	проверки только кодом, без второй модели: ровно 6 рецептов, названия не повторяются, текст по-русски, ккал столько же, сколько ингредиентов, ккал на порцию 20–1500, <code>image_query</code> 2–4 английских слова, шаги без нумерации, в <code>missing</code> нет продуктов пользователя и соли/перца/воды/сахара/масла, каждое блюдо из продуктов пользователя, несъедобное → пустой ответ, несъедобное не попало в ингредиенты, <code>exclude</code> не повторяется
<code>run.mjs</code>	запускает кейсы через тот же <code>onRequestPost</code> , что в проде (Node, без wrangler; ключи из <code>.dev.vars</code>), 4 запроса параллельно, считает токены и цену, сравнивает с прошлым прогоном (колонка «Было»), сохраняет ответы в <code>evals/results/</code> (в <code>.gitignore</code>)

Запуск:

```
npm run eval -- --runs 3           # все кейсы по 3 раза: 54 запроса, ≈ $0,95, ≈ 5 мин
npm run eval -- --only бумага --runs 5  # один кейс: центы
npm run eval -- --rescore           # перепроверить прошлый прогон после правки checks.mjs – бесплатно, без API
```

Что сделали по шагам:

- Написали кейсы и проверки, пробный прогон на 2 кейсах (\$0,02) — всё работает.
- Полный прогон 18 × 3. Разобрали провалы: три оказались ошибками самих проверок, а не модели («рисовый уксус» засчитан как «рис»; опечатки «сасиски» не находились в нормальных «сосисках»; салат на 40 ккал — правда, а не ошибка). Поправили проверки, добавили `--rescore` и сохранение ответов.
- Второй полный прогон: **463 из 468 проверок (99 %)**. Настоящие находки:
 - «**бумага, клей, сыр**» → **пустой ответ в 5 из 6 запусков**, хотя сыр есть. Модель сдавалась целиком, если рядом со съедобным лежало несъедобное. Руками не находилось: проверяли только «камни, песок, вода».
 - `exclude` иногда обходится: «Куриный плов наоборот» при уже показанном плове (2 из 6).
 - `image_query` бывает из 5 слов (≈ 10 %) — вреда нет, фото находится.
- Исправили промпт `/api/recipes` (правило теперь в SPEC выше) и сообщения голоса/фото, когда съедобного нет.
- Проверили только затронутое: бумага + клей + сыр — **5 из 5 с рецептами** (было 1 из 6); камни + яйца 3/3 без камней; камни/песок/вода, гвозди/топор, asdfgh — пустой ответ 3/3 каждый (регрессии нет).
- Коммит, push на GitHub, деплой, проверка прода curl-запросом «бумага, клей, сыр» → 6 рецептов из сыра.

Цифры: один поиск ≈ \$0,018 (Sonnet 5, \$2 / \$10 за 1M токенов), ответ ≈ 24 с (медиана); пустой ответ на несъедобное ≈ \$0,002 и 2 с. Вся работа с evals стоила ≈ \$2,30.

Грабли:

- Сначала проверять проверки: первые провалы часто — ошибка в `checks.mjs`, а не в модели. Для этого `--rescore`.
- `--only` ищет подстроку: `--only к` зацепил 12 кейсов и стоил \$0,24 впустую. Писать слово целиком.
- Прогонять каждый кейс 3–5 раз: баг «бумаги» проявлялся не всегда (1 из 6 проходил).
- Один полный прогон — эталон, дальше — точечные прогоны затронутых кейсов плюс регрессия на соседних.

Если спросят на экзамене: «Промпт — это код без компилятора, evals — его тесты. 18 кейсов, 13 проверок кодом, каждый кейс 3 раза. Прогон нашёл баг, который руками не находился: несъедобное рядом со съедобным обнуляло ответ в 5 из 6 случаев. Поправили промпт — 5 из 5, регрессии на несъедобном нет. Полный прогон стоит меньше доллара».